

Documentacion Tecnica - Proyecto Cordoba

1. Que hace el proyecto

El proyecto es una aplicacion de escritorio hecha en C++Builder con VCL. Sirve para representar una red de ciudades de Cordoba y calcular la ruta mas corta entre dos puntos.

La idea principal es usar un grafo no dirigido:

- ciudad = nodo;
- ruta = conexion;
- distancia = peso de la conexion;
- algoritmo usado = Dijkstra.

2. Archivos principales

Archivo	Para que sirve
ProyectoCordobaUI.cpp	Inicia la aplicacion.
Nodo.h	Define los datos de una ciudad.
Grafo.h/.cpp	Maneja nodos, conexiones, Dijkstra y guardado.
DatosIniciales.cpp	Carga ciudades y rutas base.
UI.h/.cpp/.dfm	Pantalla principal con mapa y busqueda.
FormNodos.h/.cpp	ABM de ciudades.
FormConexiones.h/.cpp	ABM de conexiones.
mapaCordoba3.jpg	Imagen del mapa.

3. Estructura de una ciudad

```
[cpp]
struct Nodo
{
    int id;
    char nombre[50];
    int x;
    int y;
    int xPX;
    int yPX;
    bool activo;
};
```

Los campos mas importantes son:

- id: identifica la ciudad dentro del arreglo;
- nombre: texto visible;
- xPX e yPX: posicion en el mapa;
- activo: indica si la ciudad esta disponible.

4. Como se guarda el grafo

El grafo usa una matriz:

```
[cpp]
int matriz[NODOS_CANTIDAD][NODOS_CANTIDAD];
```

El proyecto tiene:

```
[cpp]
#define NODOS_CANTIDAD 8
```

Significado de cada valor:

Valor	Significado
0	Es la misma ciudad.
> 0	Hay conexion activa.
-1	No hay conexion.
< -1	La ruta existe, pero esta cortada.

Ejemplo:

```
[cpp]
matriz[0][1] = 140;
matriz[1][0] = 140;
```

Esto significa que las ciudades 0 y 1 estan conectadas a 140 km. Se carga en ambos sentidos porque el grafo no es dirigido.

5. Ciudades iniciales

ID	Ciudad	Estado inicial
0	Cordoba	Activa
1	Villa Maria	Activa
2	Rio Cuarto	Inactiva
3	Carlos Paz	Activa
4	San Francisco	Inactiva
5	Bell Ville	Activa
6	Alta Gracia	Activa
7	Rio Tercero	Activa

Rio Cuarto y San Francisco quedan inactivas para poder darlas de alta desde el formulario.

6. ABM de ciudades

El ABM de ciudades esta en FormNodos.

Permite:

- dar de alta ciudades inactivas;
- modificar nombre o coordenadas;
- dar de baja una ciudad activa.

La baja es logica. No borra el nodo, solo cambia:

```
[cpp]
nodos[id].activo = false;
```

Asi se puede volver a activar mas adelante.

7. ABM de conexiones

El ABM de conexiones esta en FormConexiones.

Permite:

- agregar una ruta;
- modificar distancia;
- eliminar una ruta.

Para agregar o modificar se usa:

```
[cpp]
conectar(origen, destino, distancia);
```

Para eliminar:

```
[cpp]
eliminarConexion(origen, destino);
```

Eliminar deja la matriz en -1, o sea, sin conexion directa.

8. Cortar y restaurar rutas

Para cortar o restaurar se usa:

```
[cpp]
cortarORestaurarConexion(origen, destino);
```

La logica es simple:

```
[text]
140 -> -140  ruta cortada
-140 -> 140  ruta restaurada
```

Esto permite guardar la distancia original sin perderla.

9. Dijkstra

Dijkstra busca la ruta mas corta entre origen y destino.

Usa tres arreglos:

```
[cpp]
int distancias[NODOS_CANTIDAD];
bool visitado[NODOS_CANTIDAD];
int anterior[NODOS_CANTIDAD];
```

La condicion mas importante es:

```
[cpp]
nodos[i].activo && matriz[nodoActual][i] > 0
```

Eso significa que Dijkstra solo usa:

- ciudades activas;
- conexiones activas;
- distancias positivas.

Por eso, si una ruta esta cortada y queda negativa, el algoritmo la ignora.

10. Pantalla principal

La pantalla principal permite:

- elegir origen;
- elegir destino;
- buscar ruta;
- cortar o restaurar una ruta;
- abrir ABM de ciudades;
- abrir ABM de conexiones.

El mapa dibuja:

Color	Significado
Negro	Ruta activa
Rojo	Ruta cortada
Azul	Ruta encontrada
Circulo blanco	Ciudad

11. Guardado de datos

El sistema guarda:

Archivo	Guarda
nodos.dat	Ciudades y estados.
matriz.dat	Conexiones y distancias.
Historial.txt	Busquedas realizadas.

12. Diseno visual

Se uso una paleta simple:

[text]
#B8B2AA fondo gris topo
#F6F5F4 campos claros
#312F2C textos oscuros

La idea fue que la interfaz quede basica, clara y sin sobrecargar la pantalla.

13. Pruebas basicas

Antes de entregar conviene probar:

1. Buscar ruta entre dos ciudades.
2. Cortar una ruta y verificar que quede roja.
3. Buscar una ruta alternativa.
4. Restaurar la ruta.
5. Dar de alta una ciudad.
6. Modificar una ciudad.
7. Eliminar una conexion.
8. Cerrar y abrir para verificar que se guarde.

14. Explicacion corta

El sistema usa un grafo para representar ciudades y rutas. Las distancias se guardan en una matriz de adyacencia. Para calcular la mejor ruta se usa Dijkstra. Las rutas cortadas quedan con valores negativos, entonces el algoritmo no las usa.